

Datové modelování

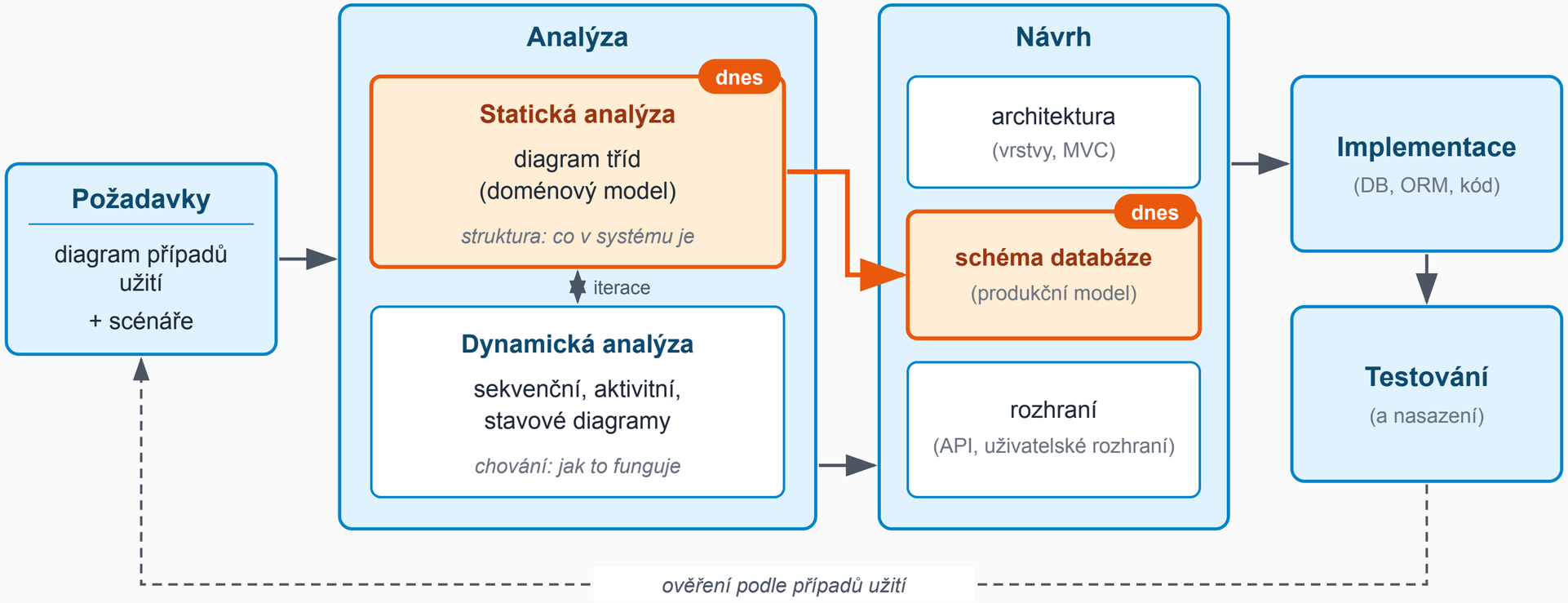
Modely a reprezentace strukturovaných dat

doc. Ing. Radek Burget, Ph.D.
burgetr@fit.vutbr.cz

Návrh informačního systému

Od případů užití k datům

Postup návrhu IS s UML



Postup je **iterativní** – jednotlivé kroky se navzájem upřesňují.

Výchozí bod: diagram případů užití

- Co už máme:
  - **Aktéři** – kdo se systémem pracuje (role uživatelů, okolní systémy)
  - **Případy užití** – co má systém aktérům umožnit
  - **Scénáře** – textový popis průběhu jednotlivých případů užití
- Co z něj zatím *nevyplývá*:
  - S jakými **daty** systém pracuje a jak spolu souvisejí
  - Jaké objekty se na případech užití podílejí a jak spolupracují
  - Jak budou data uložena a zpracována
- Odpovědi hledá **analýza** – statická a dynamická

Statická analýza

- Cíl: popsat **strukturu** modelované reality – co v systému existuje
- Vychází z případů užití a jejich scénářů
  - Podstatná jména → kandidáti na **třídy** (entity) a jejich **atributy**
  - Slovesa, vazby mezi pojmy → **vztahy** (a jejich kardinalita), operace
  - Zobecnění pojmů → **dědičnost**
- Výsledek: **doménový (konceptuální) model**
  - Diagram tříd UML, případně E-R diagram
  - Nezávislý na konkrétní implementaci (databáze, jazyk)
- Na doménový model navazuje **datové modelování** – dnešní téma

Příklad: od scénáře ke třídám

Scénář případu užití *Zápis kurzu*:

> ***Student** si v období registrace zapíše **kurz**. Každý kurz vyučuje jeden **profesor**, se kterým může spolupracovat několik **asistentů**. Kurz může mít jako **prerekvizitu** jiný kurz.*

- Třídy: *Student*, *Kurz*, *Profesor*, *Asistent*
- Vztahy: *je zapsán* (M:N), *vyučuje* (1:N), *asistuje*, *prerekvizita*
- Zobecnění: profesor i asistent jsou *zaměstnanci*
- Atributy doplníme ze scénářů a od zadavatele (jméno, login, název kurzu, kredity, …)

Dynamická analýza

- Cíl: popsat **chování** systému – jak objekty spolupracují při realizaci případů užití
- Prostředky UML:
  - **Sekvenční diagram** – výměna zpráv mezi objekty v rámci scénáře
  - **Diagram aktivit** – průběh činností, rozhodování, paralelismus
  - **Stavový diagram** – životní cyklus objektu (např. stav objednávky)
- Doplňuje doménový model
  - Odhalí chybějící třídy, atributy a vztahy
  - Doplní **operace** tříd
- Statická a dynamická analýza probíhají **iterativně**

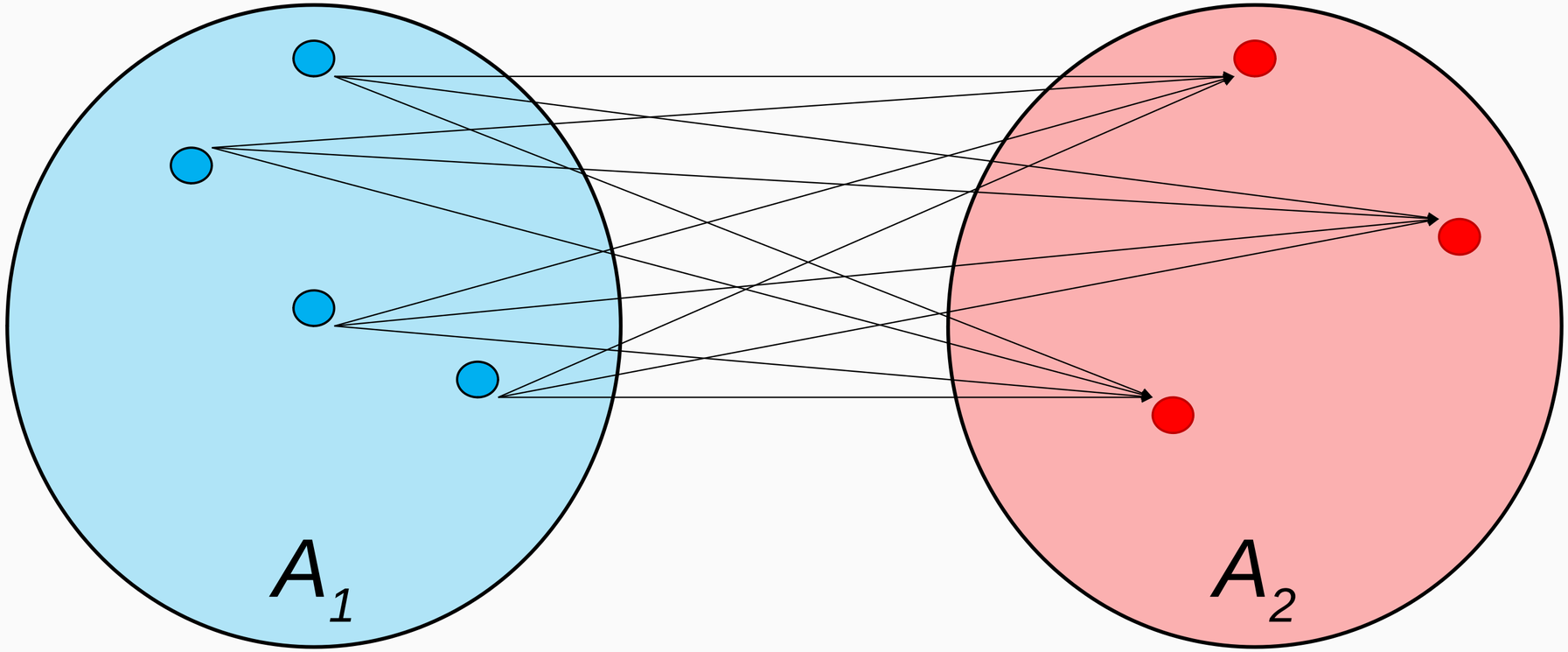
Od doménového modelu k implementaci

- Doménový model je **konceptuální** – popisuje realitu, ne implementaci
- V jednotlivých vrstvách IS z něj vznikají různé **reprezentace dat**:
  - Databázová vrstva – **schéma databáze** (produkční model, ORM)
  - Business vrstva – **objekty** v paměti aplikace a jejich operace
  - Prezentační vrstva – **vizualizace** (formuláře, tabulky) a **serializace** (webová API)
- Všechny vycházejí ze stejného modelu → potřebujeme umět:
  - Data *obecně popsat* (struktury, kolekce, vztahy)
  - Popis *transformovat* mezi různými modely

Strukturovaná data

Jak vyrobit z jednoduchých údajů složité?

Kartézský součin $A_1 \times A_2$



Výsledkem je **množina dvojic** – celkem $4 \times 3 = 12$ dvojic.

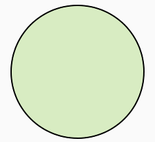
Kartézský součin

- **Uspořádaná n-tice** $(a_1, a_2, \ldots, a_n)$
- **Kartézský součin množin** $A_1 \times A_2 \times \cdots \times A_n$ je množina všech uspořádaných n-tic takových, že $a_1 \in A_1, a_2 \in A_2, \ldots, a_n \in A_n$
- Podstatné je, že i-tá složka uspořádané n-tice je prvkem množiny A_i, tj. té, která jí indexem odpovídá – na pořadí složek záleží

Struktura a kolekce

Základní typy

- **Datový typ** je *množina* všech přípustných hodnot (doména)
- **Hodnota** je *prvek* této množiny, např. $42 \in integer$, `"Novák"` $\in string$
- Složitější typy pak vytváříme operacemi nad těmito množinami (např. kartézským součinem)
- Základní nestrukturované datové typy:
  - Celočíselné
  - Reálná čísla
  - Znaky / řetězce
  - Datum / čas
  - Výčtové typy apod.



Strukturované datové typy

- Strukturovaný datový typ = datová struktura = *metadata*
  - Jak z jednodušších datových typů (ať už základních nebo i jednodušších strukturovaných) budovat složitější.
- Existují základní dva způsoby, jak strukturované datové typy vytvářet:
  - *struktura* a
  - *kolekce*.
- Vše je definováno předem, před vznikem hodnoty

Struktura

- Strukturované hodnoty vytvářené:
  - **Pevným počtem** dílčích hodnot obecně **různých** typů
  - Tedy uspořádané n-tice, které jsou prvky kartézského součinu množin dílčích datových typů
  - Hodnoty jsou pojmenované, tzn. přistupuje se k nim přes jejich unikátní jméno
  - Pojmenování složek tak nahrazuje jejich pořadí v n-tici, **není součástí dat**
- Jako synonymum pro uspořádanou n-tici (tedy hodnotu) je velmi často užíván termín *struktura* nebo *záznam*. Jako synonymum pro kartézský součin (tedy datový typ) budeme často používat *typ struktura* nebo *typ záznam*.

Schéma struktury



Příklad datového typu struktura

```
structure FyzOsoba
   properties
      UplneJmeno: string
      Jmeno:      string
      Prijmeni:   string
      DatumNaroz: date
end structure
```

Zápis ve formě kartézského součinu:

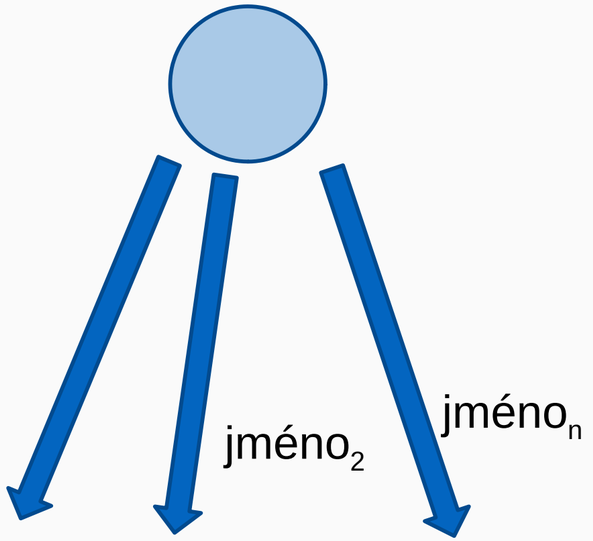
$$FyzOsoba = string \times string \times string \times date$$

Hodnota struktury

Uspořádaná čtveřice:

$$(\texttt{"Prof. Ing. Jan Novák, CSc."}, \texttt{"Jan"}, \texttt{"Novák"}, \texttt{24.5.1954}) \in FyzOsoba$$

jména vlastností	**hodnoty vlastností**
UplneJmeno:	"Prof. Ing. Jan Novák, CSc."
Jmeno:	"Jan"
Prijmeni:	"Novák"
DatumNaroz:	24.5.1954



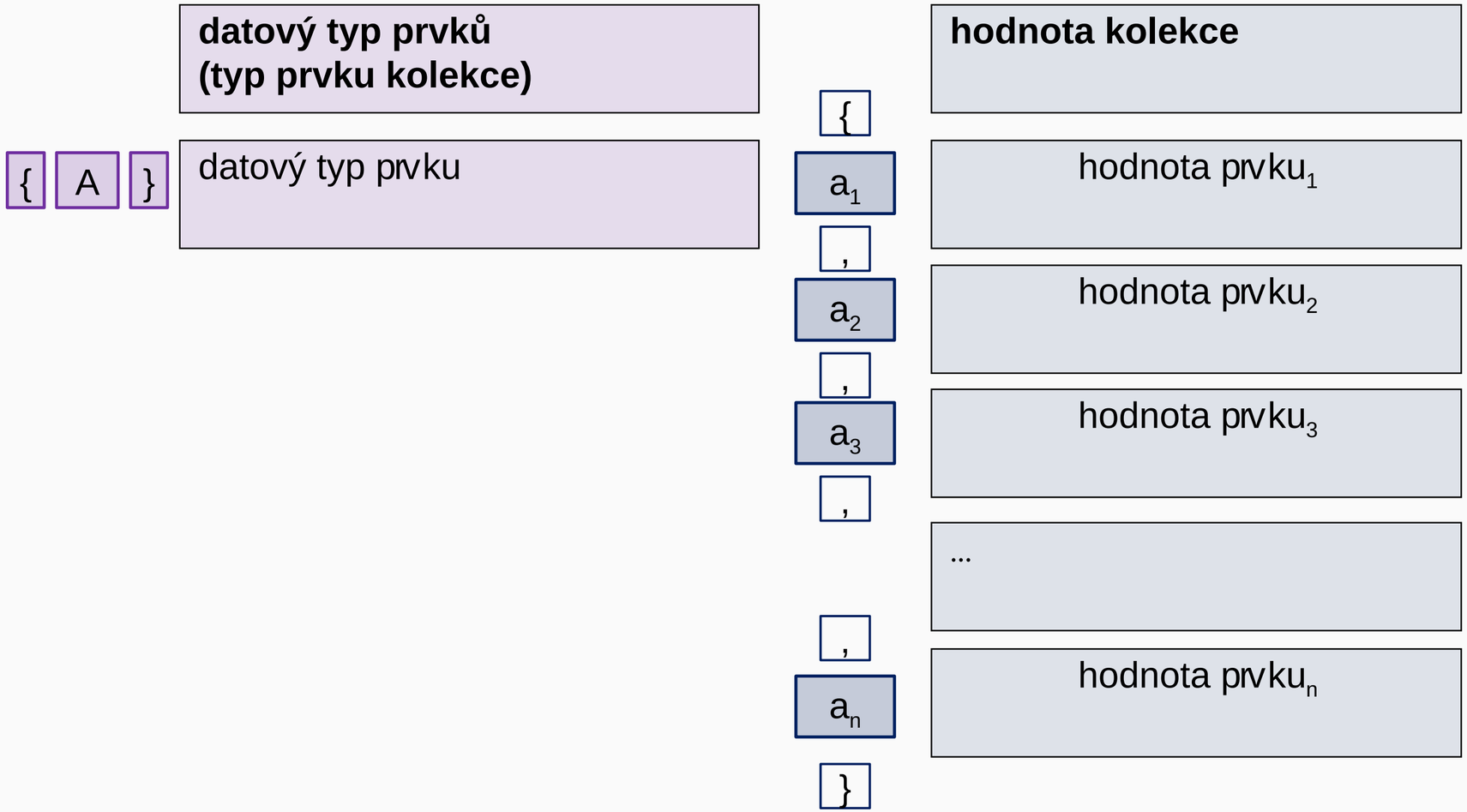
Kolekce

- **Kolekce** (synonyma jsou *posloupnost*, *seznam*) je, na rozdíl od struktur, tvořena
  - **Předem neomezeným počtem** hodnot **stejného** datového typu.

Kolekce

- Množina obsahuje obvykle každý prvek pouze jednou. Pokud je povoleno, aby daný prvek byl v množině vícekrát, mluvíme o **multimnožině**
- Tradiční seznam je *uspořádanou multimnožinou*
- Obecně lze vytvářet kolekce s prvky libovolných datových typů.
- Časté omezení je vytvářet pouze kolekce s prvky datového typu struktura

Schéma kolekce



Příklad datového typu kolekce struktur

```
collection FyzickeOsoby of
   structure FyzOsoba
      properties
         UplneJmeno: string
         Jmeno:      string
         Prijmeni:   string
         DatumNaroz: date
   end structure
```

Datový typ: množina všech podmnožin (potenční množina)

$$FyzickeOsoby = \{M \mid M \subseteq FyzOsoba\} = \mathcal{P}(FyzOsoba)$$

Hodnota kolekce

$\{o_1, o_2, \ldots, o_n\} \in FyzickeOsoby$, kde $o_i \in FyzOsoba$

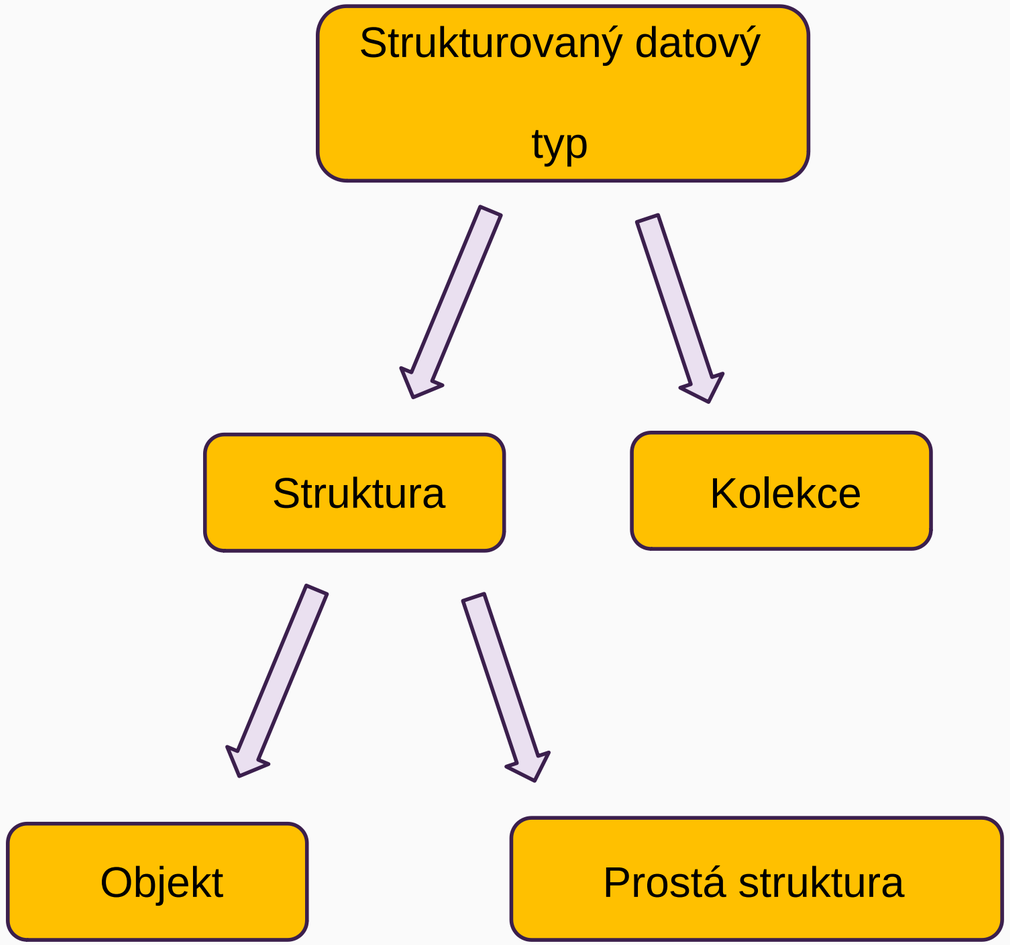
Agregáty

- Vlastnostmi kolekce jsou nejčastěji **agregáty** (agregované hodnoty), což jsou hodnoty statisticky popisující prvky kolekce nejčastěji číselných hodnot.
  - počet prvků,
  - maximum,
  - minimum,
  - součet hodnot,
  - průměr atd.

Objekt a prostá struktura

- Objekt je **struktura s identifikací**.
- Každému objektu v systému přiřazena jednoznačná identifikace nazývaná **OID** (*object identification*).
- Objekt je tedy struktura, jejíž systémovou a obvykle první vlastností je OID. Hodnotu OID generuje databázový systém při vzniku objektu a po celou dobu činnosti ji nemění.
- Tím, že má objekt OID, je *identifikovatelný* a tudíž i *odkazovatelný*. Má to za následek, že může figurovat jako člen ve vztazích. To struktura bez identifikace nemůže. Takovou strukturu bez OID budeme nadále nazývat **prostou strukturou**.

Strukturované datové typy

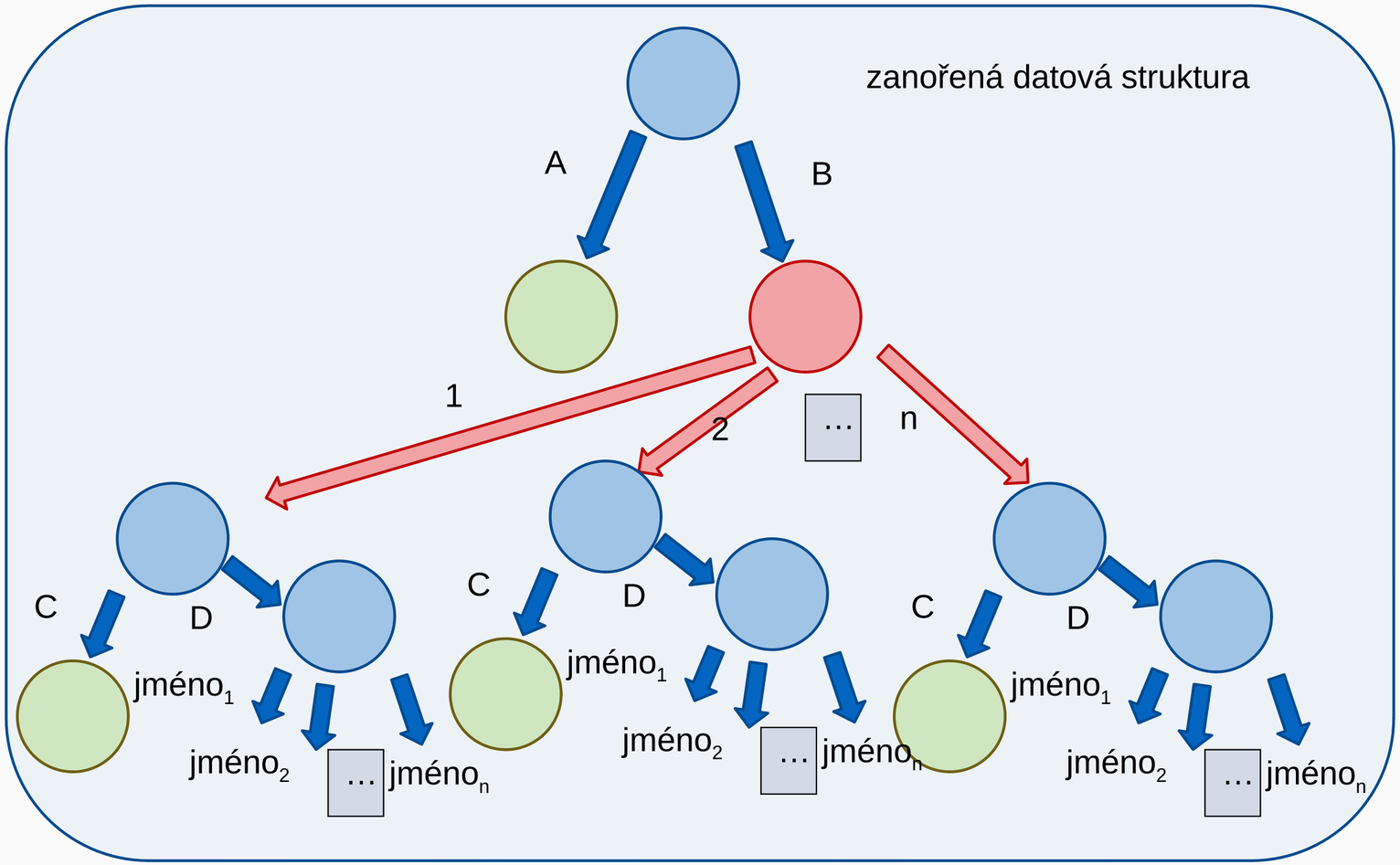


Zanořené kolekce a struktury

- Obecně lze struktury a kolekce libovolně vzájemně vnořovat

```
structure ZANORENA
   properties
      A: integer
      B: collection of structure
            properties
               C: integer
               D: structure
                     …
                  end structure
         end structure
end structure
```

Graf hodnoty zanořených typů



Datové modelování

Od struktury a kolekce k modelům

Data, metadata, atd.

- **Data**
  - Konkrétní hodnota (*výskyt*)
  - Např. "Jan Novák", "1250 Kč", … (i strukturované hodnoty)
- **Metadata** = jaký tvar mají data
  - Formální popis struktury dat
  - Záleží na datovém modelu:
    - Relační: definice struktury tabulek (relací)
    - Objektový: definice tříd
    - …

Další úrovně

- Každou úroveň metadat lze opět popsat
- **meta-metadata = meta^{2}data** = jak vypadají metadata
  - Popis datového modelu
  - Např. relační: relace je kolekce struktur, SQL jako prostředek definice metadat, …
- **meta^{3}data**
  - Jak se popisuje databázový model – kolekce, struktura, …
- … atd.

Cíle modelování

- Zobrazit modelovanou realitu
  - Zjednodušení reality pro potřeby návrhu IS – abstrakce
  - Konzultace s odborníky na cílovou doménu – spolupráce
- Vytvořit popis pomocí prostředků příslušného datového modelu
  - Vytvořit metadata – implementace
  - Různé prostředky podle způsobu použití
- Podklad pro implementaci **všech vrstev** IS

Databázové modely

- Modely, které je schopen interpretovat systém řízení báze dat (SŘBD)
- Jinak též zvané **produkční modely**
- V jejich definičním jazyku musejí být zapsána metadata pro všechny datové struktury uložené v databázi
- Prozatím budeme uvažovat jako produkční *relační a objektový datový model*.

Konceptuální modely

- Slouží pro komunikaci mezi návrháři, případně se zákazníky
- Jsou dostatečně přesné, aby byly **převoditelné** na produkční modely
- Často jsou grafické pro větší přehlednost
- Nejběžnější konceptuální modely **diagram tříd (UML)** a **E-R diagram**

Transformace mezi datovými modely

- Slouží nejčastěji pro transformaci konceptuálních modelů na produkční.
- Transformace je tím složitější, čím jsou modely více sémanticky odlišné.
- Nejčastěji se uvažuje **transformace E-R diagramu na relační datový model**.

Konceptuální modely

Entity-relationship (E-R) diagram

- Hlavní komponenty E-R diagramu jsou:
  - **entita** a
  - **vztah** (*relationship* nikoliv *relation*).
- Entity modelují objekty, které se vyskytují v modelovaném fyzickém systému
  - např. studenti, profesoři, předměty na vysoké škole
  - a jejich **atributy**
- Vztahy modelují spojení mezi entitami – například profesoři *učí* předměty.
- Navíc tvoří důležitou část E-R specifikace *integritních omezení* na entitách a vztazích, např. profesor učí pouze jeden předmět v daném čase.

Příklad E-R diagramu

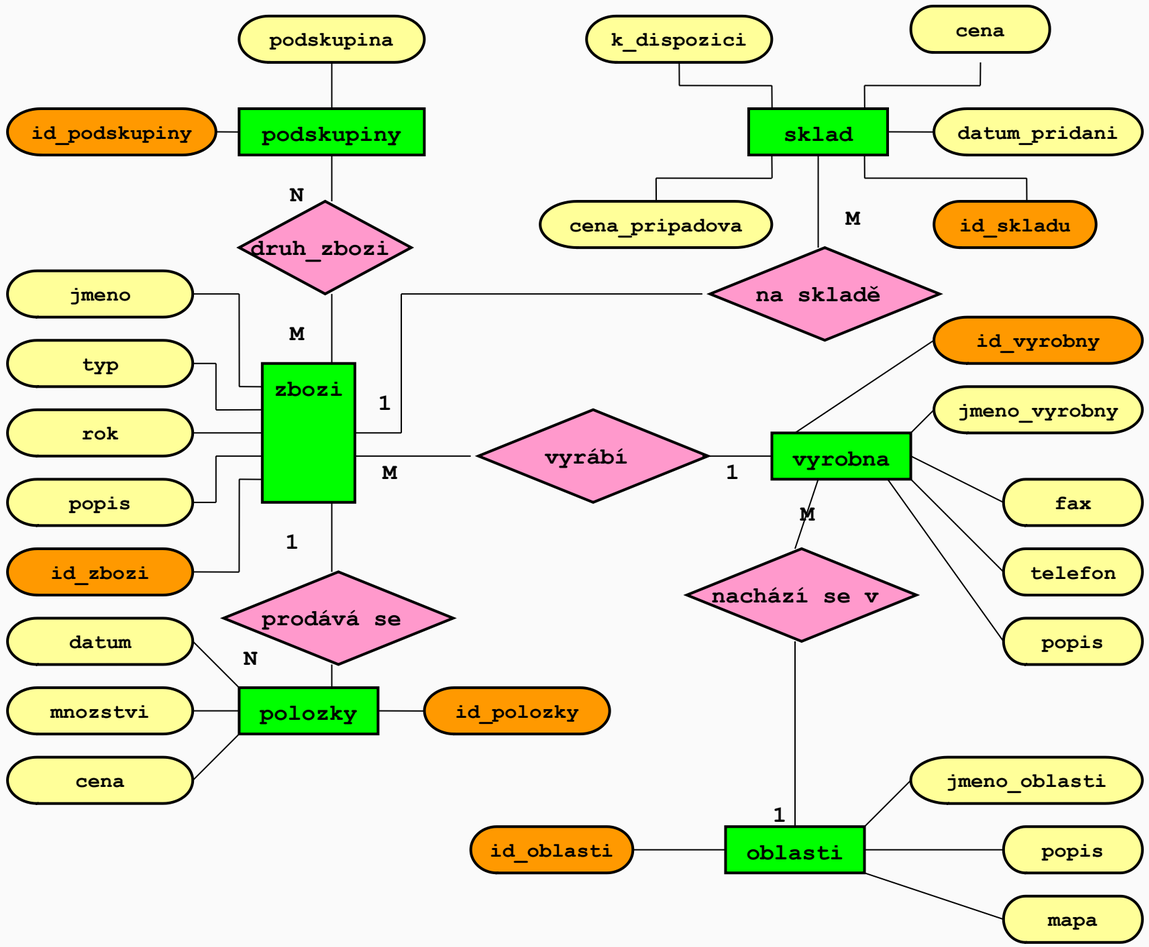
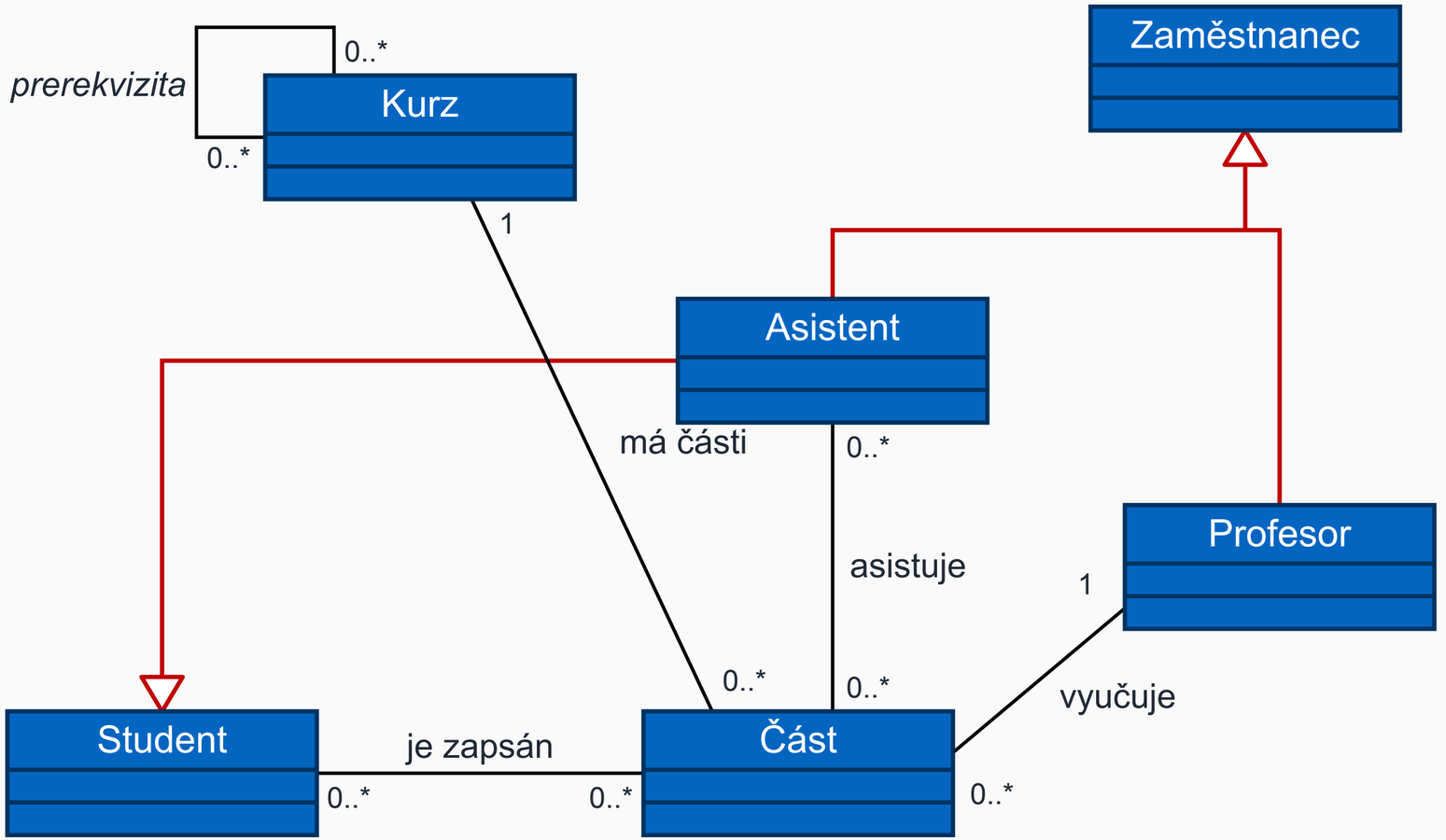


Diagram tříd

- **Třídy**
  - Jejich vlastnosti (jméno a datový typ)
- **Vztahy**
  - S různou kardinalitou
- **Dědičnost**
  - Generalizace – specializace

Příklad diagramu tříd



Databázové modely

Relační model dat

- Tabulka (= **relace**) v relačním modelu je **kolekcí struktur**, přičemž datové typy vlastností jsou *jednoduché* (tedy především ne odkazy/vztahy)
- Srovnej: *Podmnožina kartézského součinu*
  - Relace je *množina* (bez duplicit a bez pořadí), SQL tabulka je však *multimnožina*

```
collection of
   structure
      properties
         jméno_vlastnosti_1: jednoduchý_datový_typ_1
         jméno_vlastnosti_2: jednoduchý_datový_typ_2
         …
         jméno_vlastnosti_n: jednoduchý_datový_typ_n
   end structure
```

Vztahy

- Umožňují odkazovat z jedné (strukturované) hodnoty (vlastníka) na jinou (člen)
- Musí existovat datový typ jednoznačně identifikující (odkazující) strukturovanou hodnotu (např. OID)
- Vztah je definován prvkem vlastníka typu odkaz (reference) a členem, který je hodnotou odkazu identifikován.

Vztahy

- Relační model dat vztahy přímo neobsahuje
  - Vytváří se až v okamžiku dotazování (JOIN apod.)
  - (Neplést s referenční integritou!)
- Objektový model
  - Vztahy lze tvořit pomocí OID

Objektový model dat

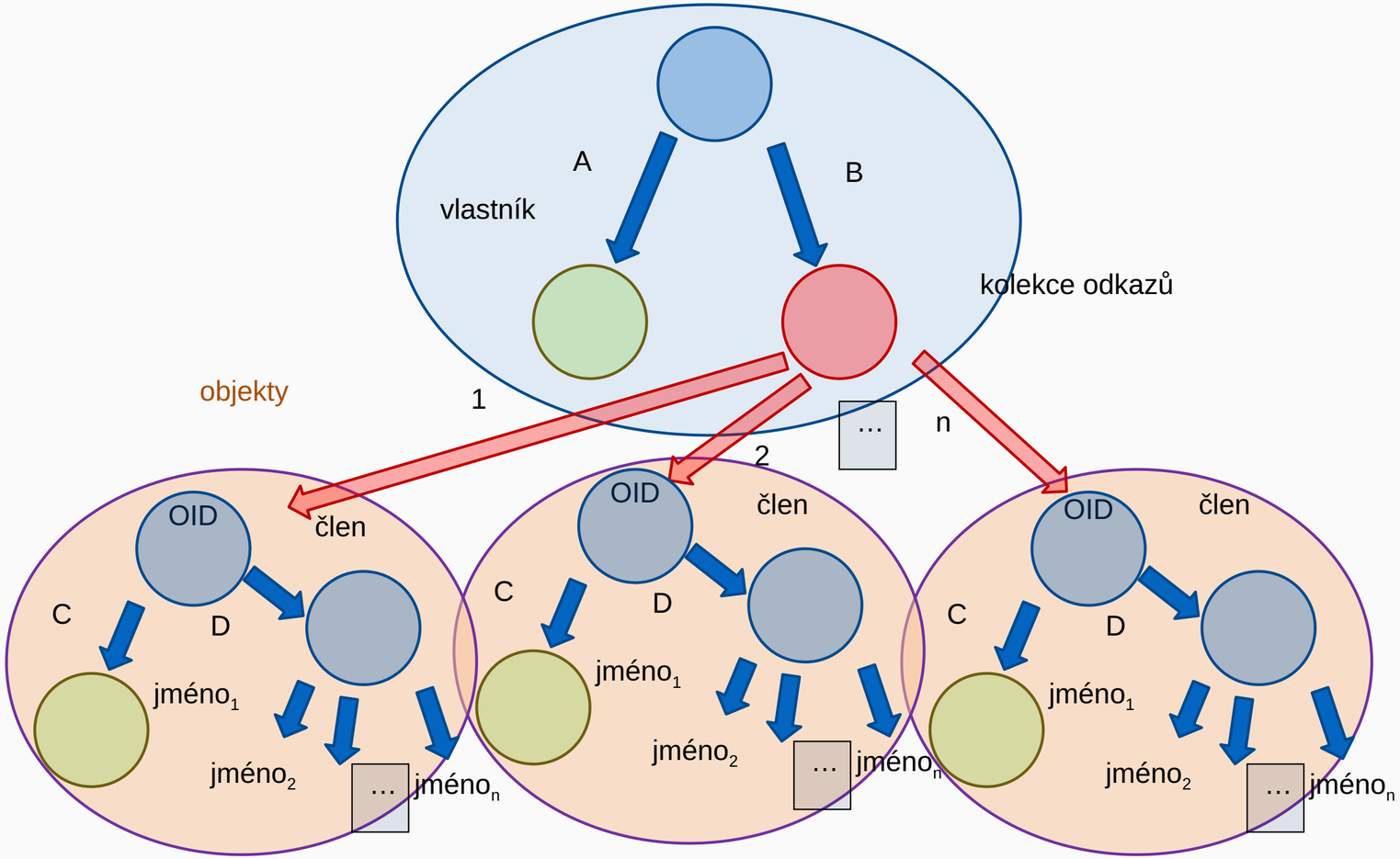
- Základní typy + datový typ OID
- Objekt je vždy strukturou na nejvyšší úrovni
- Dva druhy neomezeně zanořených struktur
  - Kolekce (někdy omezení pouze na kolekce prostých struktur a OID)
  - Prosté struktury (ostatní)
- Další vlastnosti zde neřešené (dědičnost apod.)
- Při použití objektové databáze **odpadá nutnost transformace** objektového modelu na schéma relační databáze

Odkazované struktury (objekty)

```
structure VLASTNIK
   properties
      A: integer
      B: CLEN
end structure

object CLEN
   properties
      C: integer
      D: structure
            …
         end structure
end object
```

Graf hodnoty odkazovaných typů



Transformace modelů

- Mezi modely mohou existovat transformace, zejména, pokud jsou si *sémanticky blízké*
- Velmi častou je transformace E-R diagramu na relační datový model
  - viz postup z IDS

Jiné transformace

- Diagram tříd lze transformovat na relační model obdobně jako E-R diagram
  - Je nutno řešit **dědičnost**
    - různé strategie, podrobně v Pokročilých informačních systémech
- Pro objektový databázový model lze přímo použít diagram tříd
  - V modelu se přímo definují třídy a jejich vlastnosti
  - Vztahy jsou reprezentovány vlastnostmi tříd
  - Odkazy na jiné objekty
    - např. `zbozi.vyrobce`

Objektově-relační mapování (ORM)

- Čistě objektové databáze se v praxi vyskytují minimálně
- Relační databáze jsou oproti tomu rozšířené, výkonné a odladěné
- ORM je databázová vrstva zajišťující automatické mapování objektů na relace a zpět
  - Program pracuje s objekty (např. uživatelé, smlouvy, …)
  - ORM vrstva vytváří SQL dotazy, transformuje data
- Zajišťuje konzistenci aplikace a schématu databáze
  - Možnost automatické tvorby schématu databáze apod.
  - Snazší úpravy aplikace

ORM řešení

- Java
  - Standard Jakarta Persistence (JPA), implementace např. Hibernate
- PHP
  - Např. Doctrine (Symfony), Eloquent (Laravel)
  - Použití do značné míry shodné s JPA
- Obdobně na dalších platformách
  - .NET, Python, JavaScript, …
- Více v předmětu Pokročilé informační systémy

Od modelu k aplikaci

Jak se model promítne do vrstev IS

Jeden model – více reprezentací

Příklad: kurz napříč vrstvami

Vrstva	**Kurz**	**Vztah *je zapsán* (M:N)**
Doménový model	třída *Kurz*	asociace Student – Kurz
Databáze	tabulka `kurz`	vazební tabulka `zapis`
Business	objekt `Kurz`, operace	kolekce `$kurz->studenti`, *zapsat()*
API	objekt JSON	odkaz `/api/kurzy/42/studenti`
Uživatelské rozhraní	detail, formulář	seznam studentů, tlačítko *Zapsat*

Závěr

- Ze struktur, kolekcí, základních typů a odkazů lze sestavit konceptuální i produkční modely
- Konceptuální model → **transformace** → produkční model (ručně nebo ORM)
- Doménový model určuje podobu všech vrstev IS:
  - schéma databáze, entity a operace business vrstvy, uživatelské rozhraní i API
- Co dále?
  - **Vizualizace** (2D) a **serializace** (1D) dat
  - Webová API, klientská část IS
  - Integrita, konzistence a transakce

A to je vše!

Dotazy?